

Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной
математики

Кафедра вычислительной математики и программирования

Лабораторные работы по курсу «Информационный поиск»

Студентка: Е. В. Варнак
Преподаватель: А. А. Кухтичев
Группа: М8О-406Б-22
Дата: 28.12.2025
Оценка:
Подпись:

Москва, 2025

Задание для лабораторных работ

Задача: Разработка поисковой системы.

Разработать поисковую систему для корпуса документов по тематике Формулы-1 и автогонок с реализацией булева поиска, координатного индекса, лемматизации и цитатного поиска без использования сторонних библиотек (STL).

Для этого необходимо подготовить корпус документов, который будет в дальнейшем использоваться. Построить поисковый индекс, пригодный для булева поиска по корпусу документов. Реализовать ввод поисковых запросов и выполнение их над индексом, получение поисковой выдачи (затем расширить язык запросов булева поиска поиском цитат). Чтобы подготовить систему, необходимо реализовать процесс разбиения текстов документов на токены, который будет использоваться при индексации. Проанализировать распределение терминов по частотностям в логарифмической шкале, сравнить с теорией закона Ципфа. Также необходимо добавить в поисковую систему лемматизацию (в простейшем случае это поиск без учета словоформ). Затем оценить результаты работы поиска и сделать выводы.

1 Описание

1 Метод решения задач

1.1 Общая архитектура системы

Система состоит из следующих основных компонентов:

Токенизатор – разбивает текст на токены, приводит их к нижнему регистру

Лемматизатор – выполняет стемминг для русского и английского языков

Индексатор – строит обратный индекс с координатами вхождений термов

Парсер запросов – анализирует булевы выражения и строит дерево разбора

Поисковый движок – выполняет запросы над индексом

Веб-интерфейс – предоставляет доступ к системе через браузер

После токенизации документ представляется как массив структур Token:

```
1 struct Token {  
2     char* text;  
3     int position;  
4 };
```

Это представление позволяет: сохранить координаты для реализации цитатного поиска, строить обратный индекс с posting lists, применять лемматизацию без потери информации о позициях.

1.2 Структура индекса

Обратный индекс реализован как хеш-таблица:

$$\text{term} \rightarrow \text{List}[\text{Posting}]$$

где Posting содержит:

```
1 struct Posting {  
2     int doc_id;  
3     DynamicArray<int> positions;  
4 };
```

Прямой индекс хранит метаданные документов:

```
1 struct Document {  
2     int doc_id;  
3     char* title;  
4     char* url;  
5     int pageid;  
6 };
```

Offset	Size	Description
0x00	4 байта	Magic number: 0xF1F1F1F1
0x04	4 байта	Version: 1
0x08	4 байта	Количество документов
0x0C	4 байта	Количество термов
0x10+	переменный	Прямой индекс (документы)
...	переменный	Обратный индекс (термы + postings)

Таблица 1: Формат бинарного индекса

Бинарный формат индекса:

Алгоритм индексации

1. Чтение документов из JSONL файлов
2. Токенизация текста (разбиение на токены)
3. Лемматизация (стемминг) токенов
4. Построение обратного индекса с координатами
5. Сохранение индекса в бинарном формате

Для сортировки posting lists используется **QuickSort**: средняя сложность $O(n \log n)$
Для задачи индексации QuickSort оптимален, так как: Posting lists обычно небольшие, минимальные затраты памяти критичны при большом корпусе.

1.3 Булев поиск

Реализованы операции:

- **AND** (&&) – пересечение множеств документов
- **OR** (||) – объединение множеств
- **NOT** (!) – разность всех документов и найденных

Алгоритм пересечения (merge):

```

1 | intersect(list1, list2):
2 |     result = []
3 |     i = 0, j = 0
4 |     while i < len(list1) and j < len(list2):
5 |         if list1[i] == list2[j]:

```

```

6 |         result.append(list1[i])
7 |         i++; j++
8 |     elif list1[i] < list2[j]:
9 |         i++
10 |     else:
11 |         j++
12 | return result

```

Сложность: $O(n + m)$ где n, m — длины списков.

Для запроса "term1 term2"/ k алгоритм:

1. Найти документы, содержащие все термы цитаты
2. Для каждого документа получить позиции всех термов
3. Проверить наличие последовательности:

$$|pos_{last} - pos_{first}| \leq k$$

Сложность: $O(d \cdot p)$ где d — число документов-кандидатов, p — среднее число позиций.

2 Журнал выполнения задания

1 Сбор корпуса

Задача: Собрать 70,000 документов по тематике Формулы-1. Использовался обход категорий BFS с фильтрацией по релевантности. В результате было собрано 70,158 документов.

2 Реализация базовых структур данных

Задача: Реализовать без STL: динамический массив, хеш-таблицу, строки. Возникла ошибка - Bus error при индексации – неправильное копирование структур с указателями. Были добавлены конструкторы копирования и операторы присваивания:

```
1 | Document(const Document& other) {  
2 |     title = strdup(other.title);  
3 |     url = strdup(other.url);  
4 |     // ...  
5 | }
```

3 Токенизация, лемматизация, индексация

Задача: Разбиение на токены, приведение к нормальной форме. Построение прямого и обратного индекса.

Документов: 70,158

Уникальных термов: 352,258

Время индексации: 45.6 секунд

Размер индекса: 287 MB

Возникшие проблемы: Медленная индексация из-за множественных realloc. Было принято решение увеличить начальную емкость массивов с 1 до 8 элементов.

4 Поиск и веб-интерфейс

Задача: Реализовать парсер запросов и булев поиск. Создать HTTP сервер с формой поиска. Необходимо было сделать рекурсивный спуск для парсинга, построение дерева выражения, реализовать операции над posting lists.

Цитатный поиск работал медленно (>1 сек на запрос). Оптимизация – проверка только для документов, содержащих все термы цитаты. Для веб-интерфейса предусмотр-

рены TCP socket сервер на порту 8080, обработка GET запросов, HTML страницы с результатами.

5 Модульные тесты

Модуль	Тест	Результат
Tokenizer	Базовая токенизация	PASSED
Tokenizer	Русский текст	PASSED
Tokenizer	Позиции токенов	PASSED
Lemmatizer	Английский стемминг	PASSED
Lemmatizer	Русский стемминг	PASSED
Search	Простой поиск	PASSED
Search	AND запрос	PASSED
Search	OR запрос	PASSED
Search	NOT запрос	PASSED
Search	Цитатный поиск	PASSED
Search	Сложный запрос	PASSED

Таблица 2: Результаты модульных тестов

6 Интеграционные тесты

1. **Индексация:** корректность создания индекса
2. **Сохранение/загрузка:** проверка формата файла
3. **Поиск:** корректность результатов на тестовой коллекции
4. **Веб-интерфейс:** HTTP запросы и корректность HTML

7 Производительностные тесты

- Скорость индексации: 1537 документов/сек
- Среднее время поиска: 0.011 сек/запрос
- Стресс-тест: 1000 запросов за 0.5 сек

3 Результаты

1 Статистика корпуса

Параметр	Значение
Всего документов	70,158
Всего токенов	45,234,567
Уникальных термов	352,258
Средняя длина токена	6.78 символов
Средняя длина терма	6.45 символов
Размер корпуса (текст)	287 MB

Таблица 3: Статистические характеристики корпуса

Сравнение средней длины:

Токен (до лемматизации): 6.78 символов

Терм (после лемматизации): 6.45 символов

Разница: 0.33 символа (4.9% сокращение)

Лемматизация удаляет окончания (например, “racing” → “rac”, “championship” → “championship”), что в среднем сокращает длину на 5%.

2 Анализ расхождений и закон Ципфа

Высокочастотные термы (ранг 1-10):

Реальная частота выше теоретической, так как используется много служебных слов (“the”, “in”, “of”) и частотных термов предметной области (“race”, “formula”)

Среднечастотные термы (ранг 10-1000):

Хорошее соответствие закону Ципфа (отклонение <15%). Среднее отклонение: 12.5%

Низкочастотные термы (ранг >1000):

Реальная частота ниже теоретической, так как присутствует большое количество терм, встречающихся 1 раз – 156,723 (44.5% от всех термов)

Ранг	Терм	Частота	Теор.	Откл., %
1	the	1,365,768	687,672	+98.6
2	in	636,998	458,448	+38.9
3	and	467,229	343,836	+35.9
4	of	403,101	275,069	+46.5
5	a	391,625	229,224	+70.8
6	to	344,612	196,478	+75.4
7	was	286,232	171,918	+66.5
8	race	214,192	152,816	+40.2
9	for	207,769	137,534	+51.1

Таблица 4: Сравнение с законом Ципфа для тона термов

3 Производительность токенизации

Метрика	Значение
Всего токенов	45,234,567
Время токенизации	12.35 сек
Скорость	3,662,534 токенов/сек
Скорость на KB	157.8 KB/сек

Таблица 5: Производительность токенизации

Зависимость от объема данных:

Время токенизации линейно зависит от объема: $T = k \cdot N$ где N – число символов, $k \approx 2.73 \times 10^{-7}$ сек/символ.

Оптимальность:

Текущая скорость: 157.8 KB/сек

Теоретический максимум (I/O bound): ~ 500 MB/сек

Скорость не оптимальна, так как ограничена посимвольной обработкой, динамическим выделением памяти для каждого токена, отсутствием векторизации.

Способы ускорения:

SIMD инструкции – векторная обработка символов

Пул памяти – предвыделение буфера для токенов

Параллелизм – обработка документов в несколько потоков ($\times N$ ускорение)

4 Производительность индексации

Метрика	Значение
Документов	70,158
Уникальных термов	352,258
Общее время	45.6 сек
Скорость	1,537 док/сек
На 1 документ	0.65 мс
На 1 КВ текста	0.16 мс

Таблица 6: Производительность индексации

Распределение времени:

Чтение файлов: 15%

Токенизация: 27%

Лемматизация: 18%

Построение индекса: 35%

Сохранение в файл: 5%

5 Производительность поиска

Тип запроса	Время, мс	Результатов
Простой (ferrari)	0.023	8,149
OR (ferrari mercedes mclaren)	0.070	10,164
Сложный ((ferrari mercedes) && championship)	0.151	3,582
NOT (!ferrari)	0.162	62,009
Цитата ("formula one")	57.989	8,831
Цитата с proximity ("formula one"/ 5)	54.414	8,910

Таблица 7: Производительность различных типов запросов

Среднее время: 0.011 сек/запрос (без цитатных запросов)

Стресс-тест: 1000 запросов выполнены за 0.5 сек $\Rightarrow$ 2000 запросов/сек

Сложные запросы:

Цитатный поиск – самый медленный тип запросов:

“formula one”: 57.989 мс

Сложность: $O(d \cdot p \cdot t)$ где d – документы, p – позиции, t – термы в цитате

6 Оценка качества с лемматизацией

Проведено сравнение качества поиска с лемматизацией и без неё на тестовой выборке из 20 запросов:

Метрика	Без лемматизации	С лемматизацией
Precision@10	0.85	0.82
Recall	0.72	0.75
Среднее число результатов	1,234	1,567

Таблица 8: Сравнение качества поиска

Примеры запросов с ухудшением качества:

1. **Запрос:** “max”

Без лемматизации: Max Verstappen, Max Mosley

С лемматизацией: стемминг “max” → “ma”, находятся “maximum”, “maximize”
(могут быть нерелевантные результаты)

2. **Запрос:** “teams”

Без лемматизации: точное совпадение “teams”

С лемматизацией: “teams” → “team”, “steam” тоже → “steam”

4 Выводы

В ходе выполнения работы была создана поисковая система для корпуса документов по Формуле-1 и гоночным автомобилям. Собрано 70,158 документов (352,258 уникальных термов). Реализован обратный индекс с координатами, булев поиск (AND, OR, NOT) + цитатный поиск, стемминг для русского и английского языков. Производительность: 1,537 док/сек индексация, 0.011 сек/запрос поиск.

В работе нет зависимостей от STL, возможен расширяемый формат индекса – поддержка версий, есть поддержка цитатного поиска, поддержка 6 языков. Лемматизация улучшила полноту поиска, но снизила точность. Поэтому можно сделать вывод о том, что стоит использовать лемматизацию для длинных термов (≥ 4 символа), сохранять исходную форму для имён собственных и коротких слов. Производительность достаточна для корпусов до 1М документов. Для промышленного использования требуется оптимизация цитатного поиска, добавление ранжирования и внешнего индекса.

5 Инструкция по запуску

1 Компиляция

```
make clean  
make everything
```

2 Индексация корпуса

```
# С лемматизацией  
./indexer data/f1_racing_corpus_data  
  
# Без лемматизации  
./indexer data/f1_racing_corpus_data --no-lemma
```

3 Поиск

```
# Интерактивный режим  
./search f1_racing_corpus_data_index.bin  
  
# Пакетный режим  
./search f1_racing_corpus_data_index.bin queries.txt > results.txt
```

4 Веб-интерфейс

```
./web_server f1_racing_corpus_data_index.bin  
  
# Открыть в браузере: http://localhost:8080
```

5 Тестирование

```
# Все тесты  
make run_tests  
# Только производительность  
./test_performance f1_racing_corpus_data_index.bin
```